

# Useful questions; provably few tests

**Question:** Can we design a method that works for every secret, and prove no shorter method can always work? This is decision-tree complexity with exact-position feedback.

## Prepare and check the rules

Use two kinds of counters (mark R/B or distinct symbols), paper position strips, and three folded-paper screens. About 12 counters of each kind per group suffice if guesses are drawn as they are made; keep all records visible. A parent may write dictated guesses and scores. First score visible secret RBR against RRR (2), BRB (0), and RBR (3). Repeated colors are allowed. Give only the **total correct positions**; no wrong-place-color feedback.

Print three K-1 packets, three middle, one upper, one optional extra. K: no reading; match positions and count 0-2; eliminate impossible examples. Middle: counts to 3, optional R/B reading; compare one changed feature and complete cases. Upper: counts to 4; worst-case reasoning, symmetries, and branching. Extra: 0/1 notation and small equations; variables can be replaced by counters.

## Count the same thing throughout

A **test** is a scored query. Once exactly one secret remains, naming it is free in this investigation. Original game rules require actually submitting the secret, potentially one extra turn. Our exact optima for 2, 3, 4 positions are **2, 3, 4 tests to identify**, not claims of those same turn limits for winning submission. Stop early if a score already identifies the secret.

## A shared hour with seven children and two adults

0-10: explore materials freely. 10-15: brief launch and rules. 15-35: main investigation. 35-40: movement/reset; preserve work. 40-55: continue or choose an optional proof. 55-60: share and tidy. This flexible default is our proposal, not a source prescription.

One adult stays with K,K,1. The organizer alternates middle and upper, leaving a concrete next attempt and returning for an explanation. In games keep two opposing sides and rotate a checker. Proof continuations need adult attention; pages are not a completion quota.

**K-1 stop:** Build a complete two-test method. **Optional proof:** for any first test, build two secrets scoring one, so one test cannot always work.

**Middle stop:** Explain why the three-test method works. **Optional proof:** combine the one-B triple obstruction with color renaming to cover every first test. Without renaming, claim only the RRR-first obstruction.

**Upper stop:** Explain why four tests always suffice. **Optional proof:** defend the triple and four-candidate branches separately, then use symmetry to prove optimality. Choose one older group's proof conversation at a time.

# Two and three places: complete proofs

## K-1: elimination becomes a decision tree

RR scores 0 only for BB, 2 only for RR, and 1 for RB or BR. In the last branch test RB: scores 2 or 0 distinguish them. So two tests suffice. One cannot always work: for every first test, two secrets match in exactly one position. Build those counterexamples.

## Middle: a three-test upper bound

Let baseline RRR score  $s$ , the number of Rs in the secret. Flipping one test position R to B changes its contribution from 1 to 0 if the secret there is R, or 0 to 1 if it is B. The new score is  $s - 1$  or  $s + 1$ , never unchanged. Tests BRR and RBR reveal positions 1 and 2. Their number of Rs subtracted from  $s$  determines position 3. This procedure covers all eight secrets.

## Middle: why two tests cannot guarantee success

After RRR scores 2, candidates are BRR, RBR, RRB. The complete second-test table is:

| Test | BRR | RBR | RRB |
|------|-----|-----|-----|
| RRR  | 2   | 2   | 2   |
| RRB  | 1   | 1   | 3   |
| RBR  | 1   | 3   | 1   |
| RBB  | 0   | 2   | 2   |
| BRR  | 3   | 1   | 1   |
| BRB  | 2   | 0   | 2   |
| BBR  | 2   | 2   | 0   |
| BBB  | 1   | 1   | 1   |

Every row repeats a score; some answer leaves two candidates. Conceptually, the three secrets differ only in the position of one B. A test only reveals whether that B falls inside its B positions, giving at most two outcomes. For the numbered symmetry step, physically turn BRB into RRR by swapping R/B at places 1 and 3 in both secrets and queries. An equal pair stays equal and an unequal pair stays unequal. Every score is preserved. Do this at the B positions of any first query; it becomes RRR, so the lower bound covers all first queries and all adaptive methods.

## Hints

Make candidate rows tangible. The table is optional once a child explains the one-B structure and two possible scores; further entries add no proof. Test a troublesome pair first. Ask what score a test would earn for each candidate. A test need not itself be a possible secret to reveal useful information.

# Four tests: upper and lower bounds

## Upper bound

RRRR and three separate single-position flips determine places 1–3 by score changes; the baseline R count determines place 4. Four tests suffice. Generally this uses  $n$  tests to identify an  $n$ -bit secret, at most  $n + 1$  turns if winning submission is required.

## A difficult answer that stays consistent

By independent color renaming, take the first test to be RRRR. Answer 2, leaving six two-B secrets. Classify the second test by its B count. Position permutations and complementing a test (replacing score  $s$  by  $4 - s$ ) reduce all cases:

**0 or 4 Bs:** no information; six remain.

**1 B (BRRR):** scores 3 or 1 each leave three candidates. Take score 3: BBRR, BRBR, BRRB. Ask: *Which position is fixed? What is the only feature that changes?* They share the first B; the other B occupies one of three places. Any last test has a fixed first-position contribution and only two possible scores on the remaining one-B triple. It cannot separate all three. The score-1 branch has the same structure; 3-B tests are complementary.

**2 Bs (BBRR):** score 4 leaves BBRR, score 0 leaves RRBB, score 2 leaves BRBR, BRRB, RBBR, RBRB. Choose the four-candidate branch. For any last test with  $k$  Bs and a two-B secret, let  $j$  positions have B in both. The score is  $2 - k + 2j$ . There are at most three possible scores, so four candidates cannot all be separated.

Every second test has a consistent branch that needs more than one further test. Thus three tests cannot guarantee identification; four is optimal. The lower bound uses actual score structure, not merely five possible answers per test.

## Facilitating a hard proof

Let the child physically sort six code cards into score piles. Ask them to make the largest pile small, then play the stubborn setter. Page 1 supplies a satisfying stop. Pages 2–3 are an optional adult-supported lower-bound investigation; the final-query triple argument is different from the four-candidate counting argument. If full optimality is too much, obtain a proof of the four-test method and one failed three-test plan. Record that boundary honestly.

**Relationship to Week 3:** Here the hidden object is an ordered binary string, queries may adapt, and the goal is a worst-case measurement optimum. Score structure and adversarial lower bounds deepen earlier identification ideas.

# Five bits and a research landmark

## Extra: decode and prove

The example is **10110**, with scores 2,2,2,0. Let  $w$  count secret 1s and  $s_0, s_1, s_2, s_3$  be scores in printed order. Then  $w = 5 - s_0$ , and

$$x_4 + x_5 = \frac{s_1 - s_0 + 2}{2}, \quad x_3 + x_5 = \frac{s_2 - s_0 + 2}{2}, \quad x_2 + x_5 = \frac{s_3 - s_0 + 2}{2}.$$

Each 0-to-1 change gains one match for a secret 1 and loses one for 0. If any pair-sum is 0 or 2, it fixes  $x_5$ , then  $x_2, x_3, x_4$ ;  $w$  fixes  $x_1$ . Otherwise all three sums are 1. If  $x_5 = 0$ , the middle bits are 111 and  $w$  is 3 or 4. If  $x_5 = 1$ , they are 000 and  $w$  is 1 or 2. The ranges do not overlap. So  $w$  fixes the case and  $x_1$ .

This proves four *preselected* tests distinguish all 32 secrets. The page claims sufficiency, not an independently proved five-place optimum. Matching scores encode binary-hypercube distances; distinguishing landmarks form a resolving set.

## Undergraduate and research connections

Decision trees, adversary arguments, Hamming distance, and information bounds belong to discrete mathematics and algorithm analysis. The small optima here have explicit upper and lower proofs.

Cáceres et al., *On the Metric Dimension of Cartesian Products of Graphs* (2005 preprint; 2007 publication), §2, PDF p. 3, gives exactly our four five-bit landmarks. §6, PDF p. 8, equates exact-position static Mastermind with resolving sets in Hamming graphs. This directly adapts a finite construction from a research paper, with our elementary decoding proof.

Knuth, *The Computer as Master Mind* (1976–77), gives a five-guess strategy for commercial four-place, six-color Mastermind; see Knuth's author page and strategy link. That game also has wrong-place-color feedback. Its result does not directly apply here. The shared idea is minimizing the worst surviving candidate set.

## Teaching lineage and future use

JRMF *Crack the Code*, PDF pp. 2–4 supplies binary exact-position play and the  $n + 1$ -turn challenge. The optimum/adversary tasks and extra deepen it. *Math Circle by the Bay*, pp. ix–x supports manipulatives, varied pace, and optional challenges.

Use F06-K-v2/M-v2/U-v2/X-v2; prepared, not taught. Save more colors, noise, and average-case analysis for later. `plans/verify-week-06.py` searches all adaptive query trees for small optima and checks all 32 extra signatures.